

Capítulo 1

Certificado TLS, MQTTS

1.1. Procedimiento de configuración de TLS y WebSocket seguro

1.1.1. Creación de la infraestructura de directorios

Primero, cree el directorio donde se almacenarán los certificados de Mosquitto:

```
sudo mkdir -p /etc/mosquitto/certs
```

1.1.2. Generación de la Autoridad Certificadora (CA)

1.1.2.1. Crear la clave privada de la CA

Ejecute el siguiente comando para generar una clave RSA de 2048 bits:

```
openssl genrsa -out ca.key 2048
```

1.1.2.2. Crear el certificado de la CA

Use el siguiente comando para crear el certificado autofirmado de la Autoridad Certificadora. **IMPORTANTE:** cuando se le pida el `Common Name`, ingrese algo como `MiCA_Local`.

```
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt
```

Durante la ejecución, se solicitará información en un formato denominado *Distinguished Name (DN)*.

- **Country Name** (Código de país): CO
- **State or Province Name** (Provincia o estado): Cauca
- **Locality Name** (Localidad): Popayán
- **Organization Name** (Organización): Plc

- **Organizational Unit Name** (Unidad organizacional): Automática
- **Common Name** (Nombre común): Mcp_IA
- **Email Address** (Correo electrónico): frealpe@unicauca.edu.co

1.1.3. Generación del certificado del servidor

1.1.3.1. Crear la clave privada del servidor

En la misma carpeta de certificados, ejecute:

```
openssl genrsa -out server.key 2048
```

1.1.3.2. Crear la solicitud de firma de certificado (CSR)

Cree el archivo de solicitud de firma del certificado:

```
openssl req -new -out server.csr -key server.key
```

Ingrese la siguiente información cuando se le solicite :

- **Country Name** (Código de país): CO
- **State or Province Name** (Provincia o estado): Cauca
- **Locality Name** (Localidad): Popayán
- **Organization Name** (Organización): Servidor_MQTT
- **Organizational Unit Name** (Unidad organizacional): Automática
- **Common Name** (Nombre común): 192.168.20.23 (o la dirección IP de su servidor)
- **Email Address** (Correo electrónico): frealpe@unicauca.edu.co

Para los campos de atributos adicionales (**A challenge password** y **An optional company name**), puede dejarlos en blanco presionando Intro.

1.2. Configuración de Mosquitto con WebSocket y Seguridad

1.2.1. Introducción

Mosquitto es un servidor MQTT ligero y de código abierto que facilita la comunicación entre dispositivos IoT. Para garantizar una comunicación segura y confiable en entornos de producción, es esencial configurar Mosquitto con múltiples puertos que soporten distintos protocolos y niveles de seguridad.

Esta sección describe cómo configurar Mosquitto para soportar cuatro tipos de conexiones:

1. **MQTT estándar (puerto 1883)**: Para comunicación sin cifrado, útil en redes locales confiables.
2. **MQTT Seguro - MQTTS (puerto 8883)**: Para comunicación cifrada mediante TLS, utilizado por dispositivos IoT tradicionales.
3. **WebSocket (puerto 8083)**: Para navegadores web y aplicaciones React sin cifrado.
4. **WebSocket Seguro - WSS (puerto 8084)**: Para navegadores web con cifrado TLS, la opción más segura para aplicaciones web.

Además, implementaremos autenticación basada en usuario y contraseña, así como listas de control de acceso (ACL) para restringir qué clientes pueden publicar o suscribirse a temas específicos.

1.2.2. Archivo de Configuración Principal

El archivo de configuración de Mosquitto se encuentra typically en `/etc/mosquitto/mosquitto.conf`. A continuación se detalla la configuración completa con seguridad habilitada:

```
# =====
# Configuraci\'on Mosquitto con WebSocket + Seguridad
# =====

# --- Puerto MQTT normal (para Node.js, PLC, etc.)
listener 1883 0.0.0.0
protocol mqtt

# --- Puerto MQTT SEGURO (MQTTS - El que usa la "llave")
listener 8883 0.0.0.0
protocol mqtt
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

# --- Puerto WebSocket (para navegadores, React, etc.)
listener 8083 0.0.0.0
protocol websockets

# --- Puerto WebSocket SEGURO (WSS - Para la web con SSL)
listener 8084 0.0.0.0
protocol websockets
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

# --- Seguridad
```

```
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/aclfile
```

```
# --- Logs
connection_messages true
log_type all
log_dest syslog
log_dest stdout
```

1.2.3. Descripción de Directivas de Configuración

1.2.3.1. Listeners (Puertos de Escucha)

Cada `listener` define un puerto sobre el cual Mosquitto escucha conexiones entrantes. La dirección `0.0.0.0` indica que acepta conexiones de cualquier interfaz de red.

- **listener 1883 0.0.0.0:** Puerto MQTT estándar sin encriptación.
- **listener 8883 0.0.0.0:** Puerto MQTT seguro con soporte TLS.
- **listener 8083 0.0.0.0:** Puerto WebSocket sin encriptación.
- **listener 8084 0.0.0.0:** Puerto WebSocket seguro con TLS.

1.2.3.2. Certificados y Claves TLS

Para los puertos seguros (8883 y 8084), se deben especificar tres archivos generados anteriormente:

- **cafile:** Certificado de la Autoridad Certificadora (`ca.crt`).
- **certfile:** Certificado del servidor (`server.crt`).
- **keyfile:** Clave privada del servidor (`server.key`).

Estos archivos deben tener permisos apropiados y ser accesibles solo por el usuario de Mosquitto.

1.2.3.3. Autenticación y Control de Acceso

- **allow_anonymous false:** Desactiva las conexiones anónimas, requiriendo que todos los clientes proporcionen credenciales.
- **password_file:** Ruta al archivo que contiene los nombres de usuario y contraseñas hasheadas.
- **acl_file:** Ruta al archivo que define las listas de control de acceso para cada usuario.

1.2.3.4. Registro de Eventos (Logging)

- **connection_messages true:** Registra mensajes de conexión y desconexión de clientes.
- **log_type all:** Registra todos los tipos de eventos.
- **log_dest syslog:** Envía los logs al servicio syslog del sistema.
- **log_dest stdout:** Envía los logs a la salida estándar (útil para debugging).

1.2.4. Notas importantes

- Asegúrese de que todos los certificados y claves estén almacenados en `/etc/mosquitto/certs`.
- El **Common Name** del servidor debe coincidir con la dirección IP o el nombre de dominio del servidor MQTT.
- Los certificados generados con este procedimiento son válidos por 3650 días (10 años).
- Este proceso establece una Infraestructura de Clave Pública (PKI) para asegurar la comunicación mediante TLS.